

TECHNICAL COMPENDIUM

Digital Systems and Logic
Design

*"Leveraging Register-Transfer Level (RTL)
principles to architect and verify complex
digital systems via graphical schematic
capture and simulation."*

Nikhil Jindal

Executive Summary

This document serves as a specialized technical appendix to my professional profile. It catalogs the design, implementation, and functional verification of digital architectures developed using **CircuitVerse** and logic synthesis principles. The following sections demonstrate proficiency in translating Boolean requirements into optimized hardware structures.

CORE COMPETENCIES FOCUSED IN THIS REPORT

RTL Schematic Capture	Combinational Optimization	Arithmetic Logic Units
Sequential Machine Design	Functional Verification	Modular System Architecture

TABLE OF CONTENTS

01. Integrated 8-Bit Computational System & Custom SRAM Architecture	3rd-4th Sem
A complete RTL design featuring an 11-instruction ALU and a 256-bit hierarchical memory unit.	
	03
02. Automated Warehouse Stock Management System	5th Sem
Automating warehouse logistics using combinational decoders and sequential counters.	
	12

*Note: All circuits documented hereafter are functionally verified through exhaustive simulation in CircuitVerse.

Project 01: Integrated 8-Bit Computational System & Custom SRAM Architecture

Project Overview

Developed in my **3rd-4th Semester**, this project involves the ground-up design of a modular 8-bit computational system. The architecture integrates a high-functionality **11-Instruction Arithmetic Logic Unit (ALU)** with a custom-designed **256-bit Static RAM** unit. Every component, from the individual memory bit-cells to the complex sequential division logic, was architected from scratch to simulate a functional hardware data-path.

Key Design Metric: The system operates on a 9-bit user-input bus (8 bits for Data A/B, 1 bit for Read/Write control) and an 8-bit Memory Address bus, allowing for 256 unique storage locations.

I. System Architecture & Data Path

The system follows a synchronized RTL (Register-Transfer Level) flow. Data is latched into **4-bit PIPO (Parallel-In Parallel-Out) Registers** before being processed by the ALU or stored in memory.

- **Registers:** Dual 4-bit PIPO registers serve as temporary buffers for Data A and Data B to ensure stable inputs during high-frequency clock cycles.

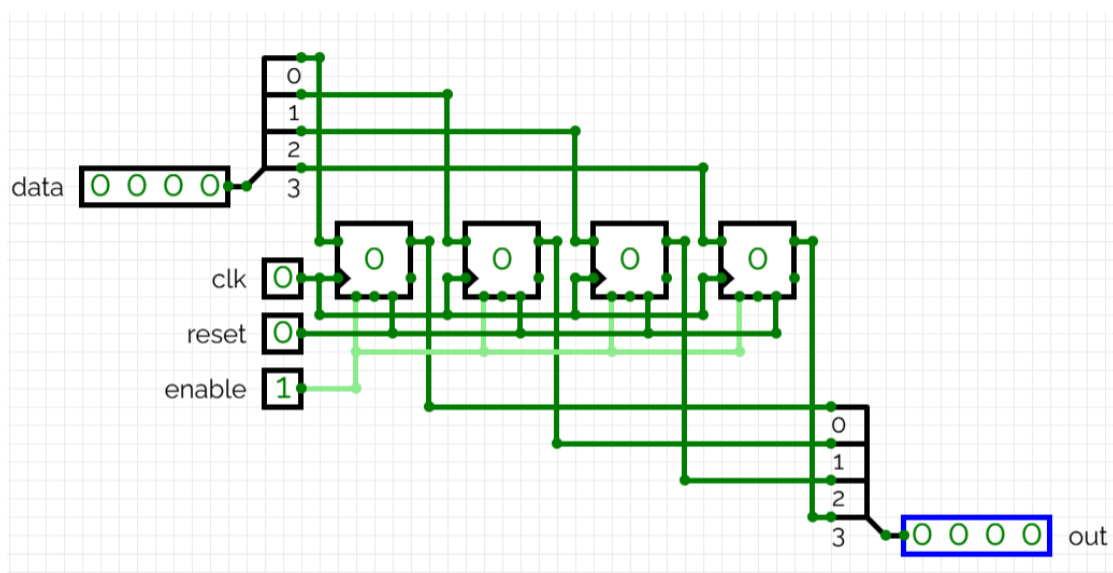


Fig: 4 bit PIPO Register

- **Control Unit:** Manages the **Clock (CLK)**, **Enable**, and **Reset** signals to synchronize data transfer between the registers, ALU, and RAM.

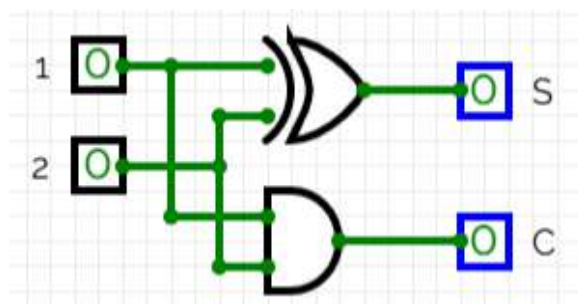


Fig: Half Adder

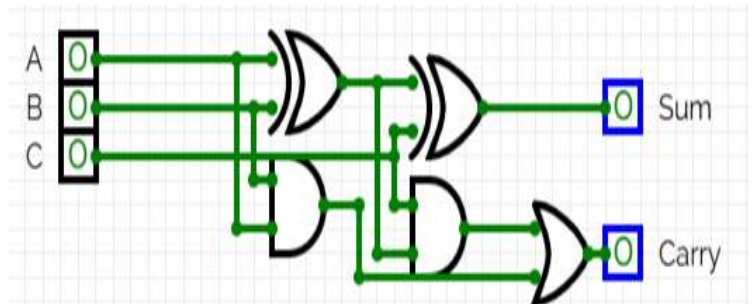


Fig: Full Adder

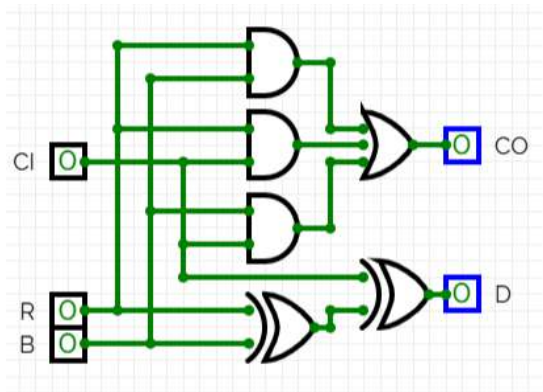


Fig: Divider Unit

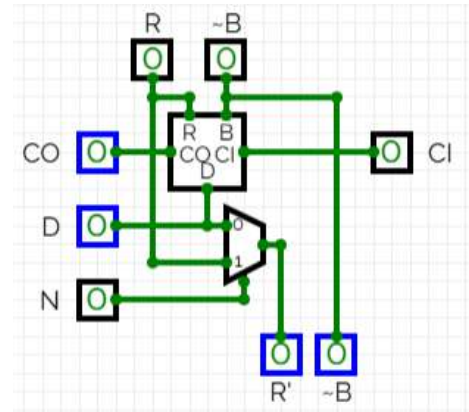


Fig: Divider Cell

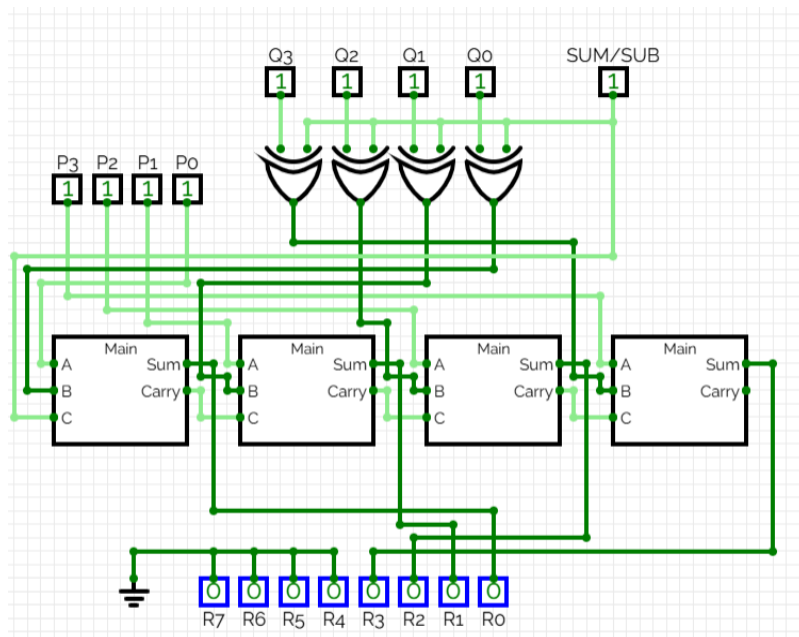


Fig: 4-bit Adder-Subtractor

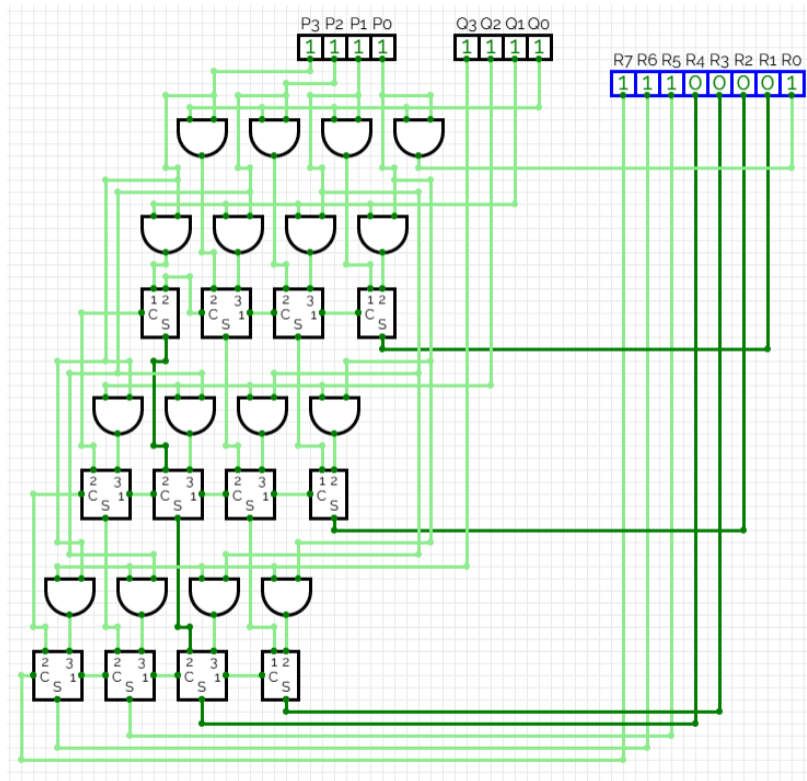


Fig: 4-bit Wallace Tree Multiplier

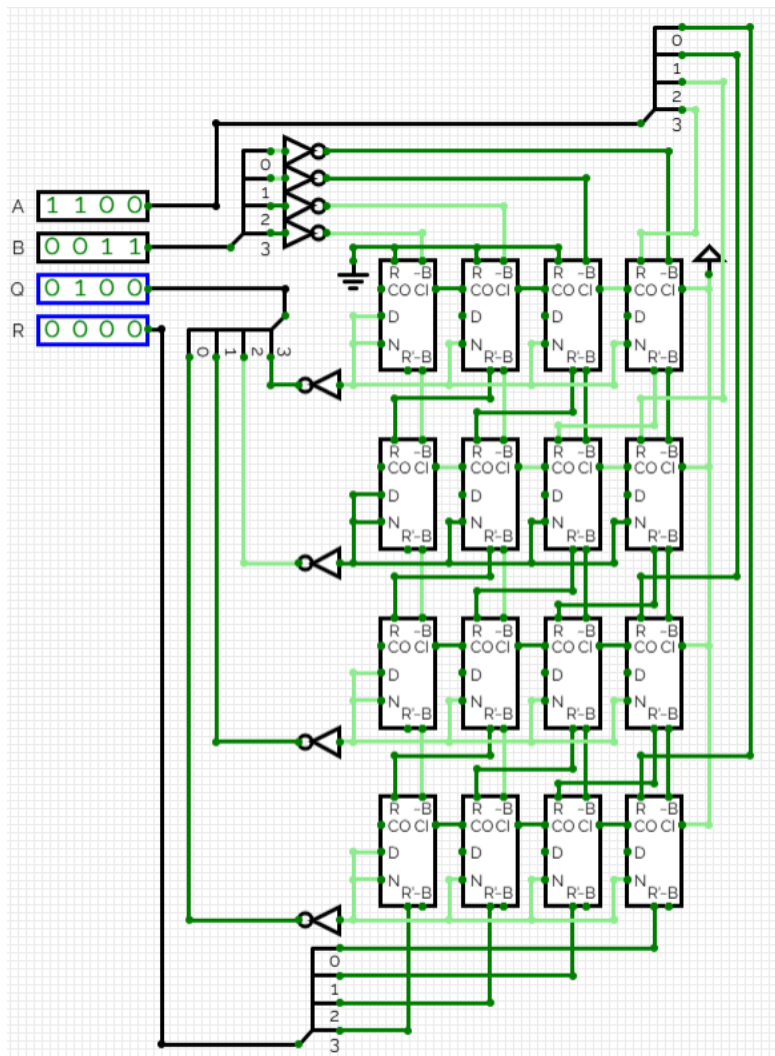


Fig: 4-bit Divider

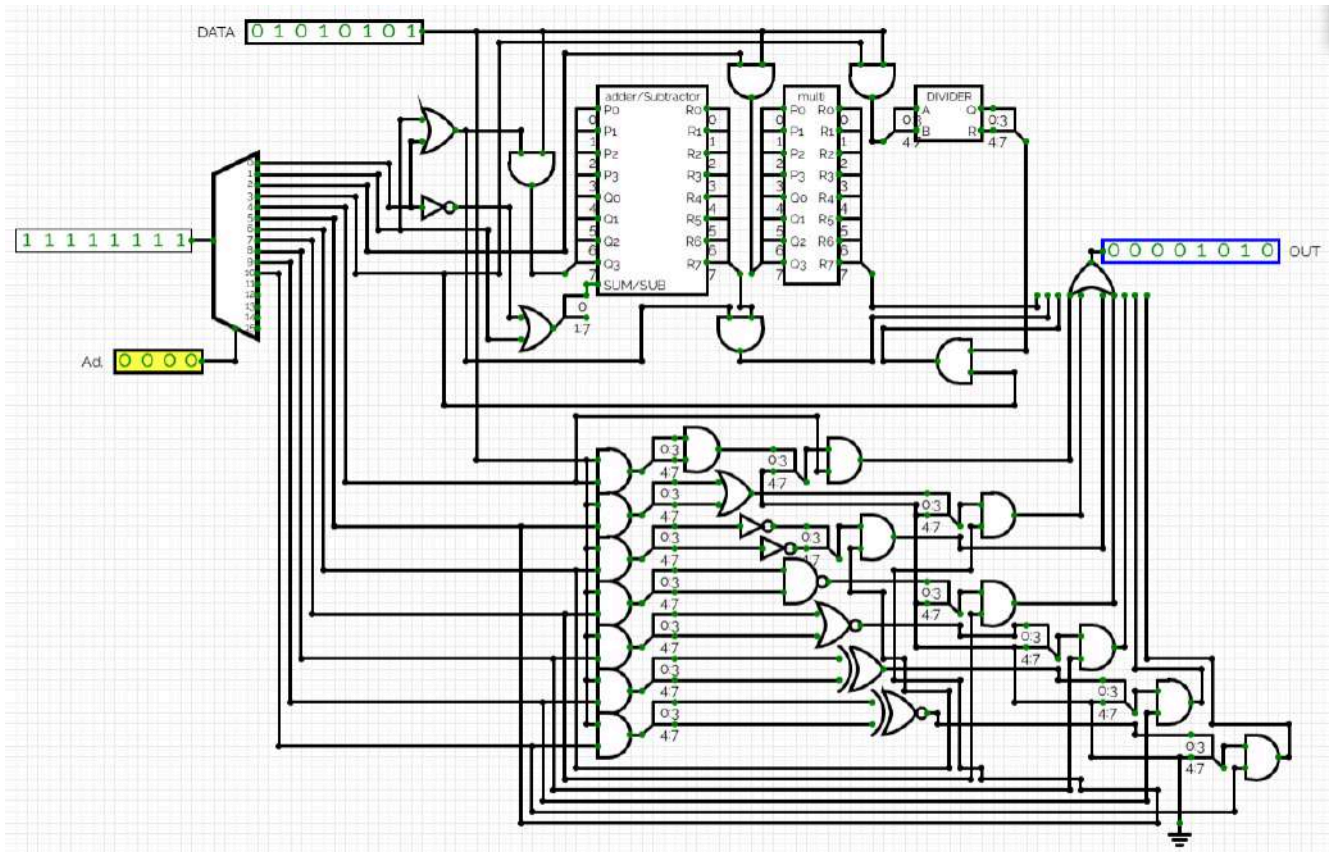


Fig: 8-Bit Alu with 11 instructions (4 arithmetic and 7 logical) accessed by 4 bit input

II. Custom 256-Bit SRAM Design (From Scratch)

Unlike standard library components, this memory unit was built hierarchically:

1. **Storage Cell:** A custom latch-based cell capable of **Read, Write, and Hold** operations.

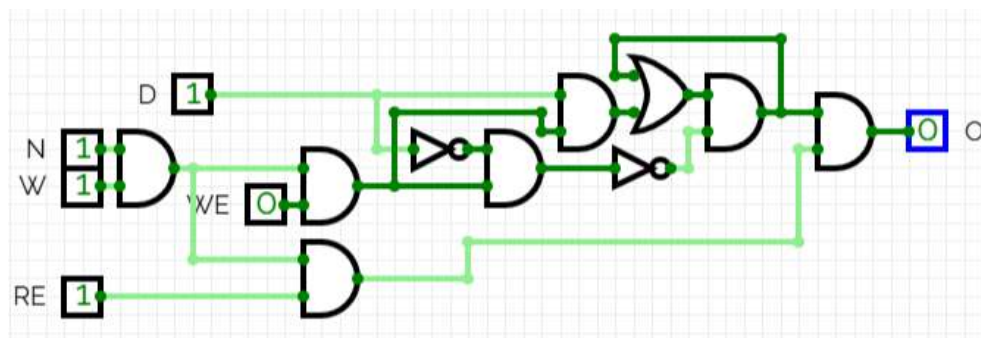


Fig: Combinational Storage Cell Unit

2. **Array Logic:** An 8-bit address of memory array manages the selection of 256 individual bit locations with 9 bit data input where (0-3 for A), (4-7 for B) and 9th bit is of read/write operation selection. An output block act as a Sense block to give output of read operation.

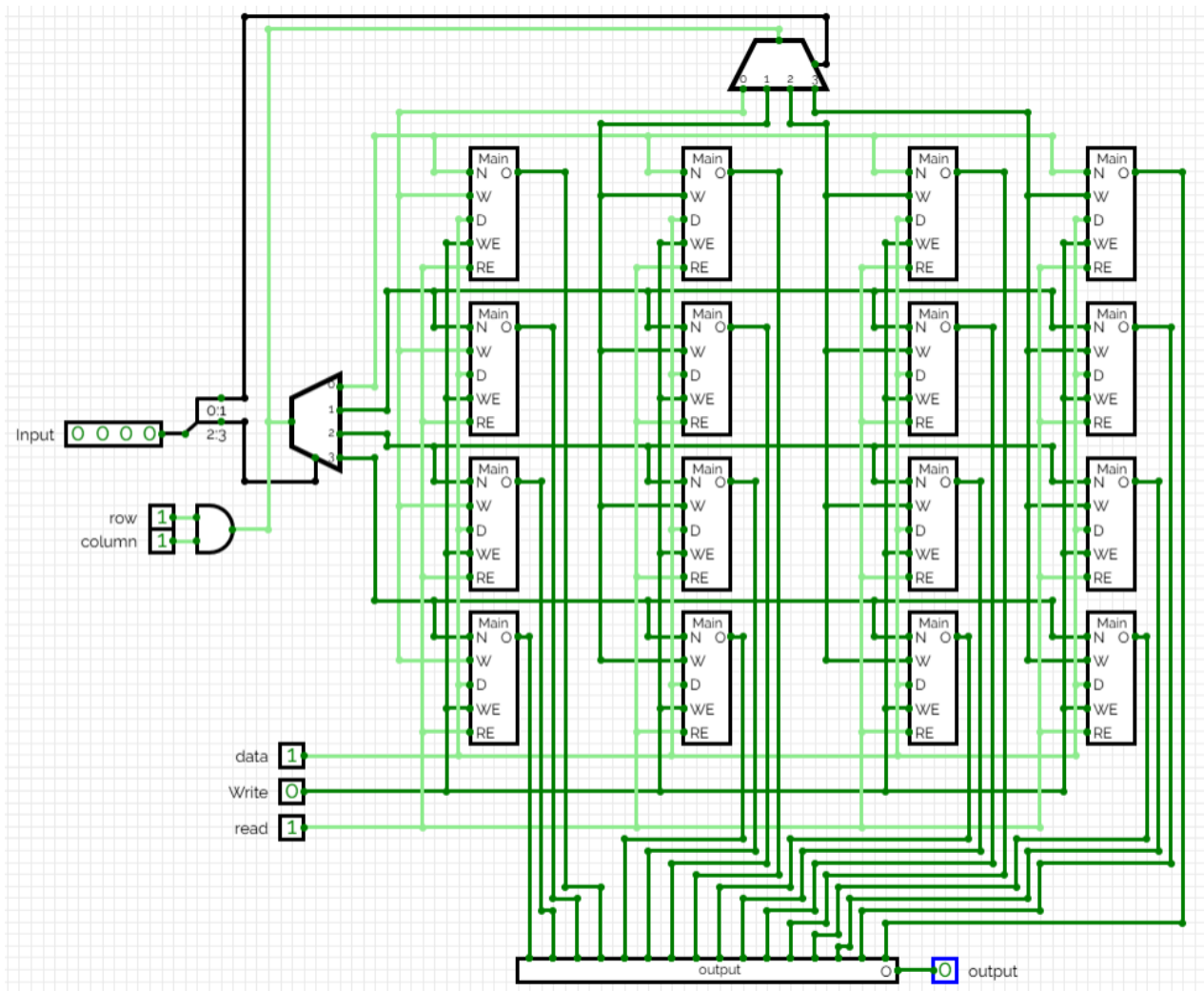


Fig: 16 bit Memory Unit

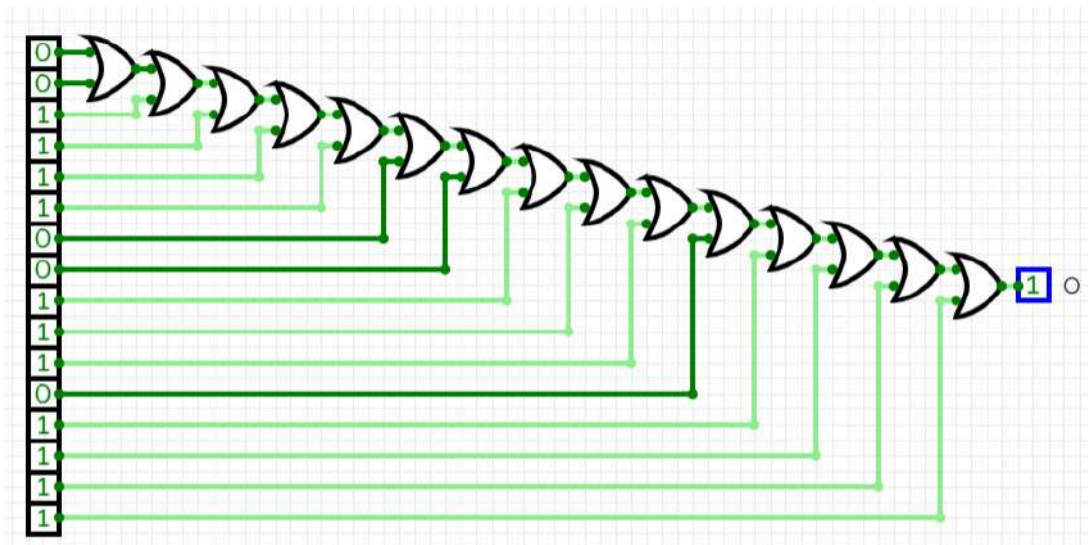


Fig: 16 Bit SRAM Output

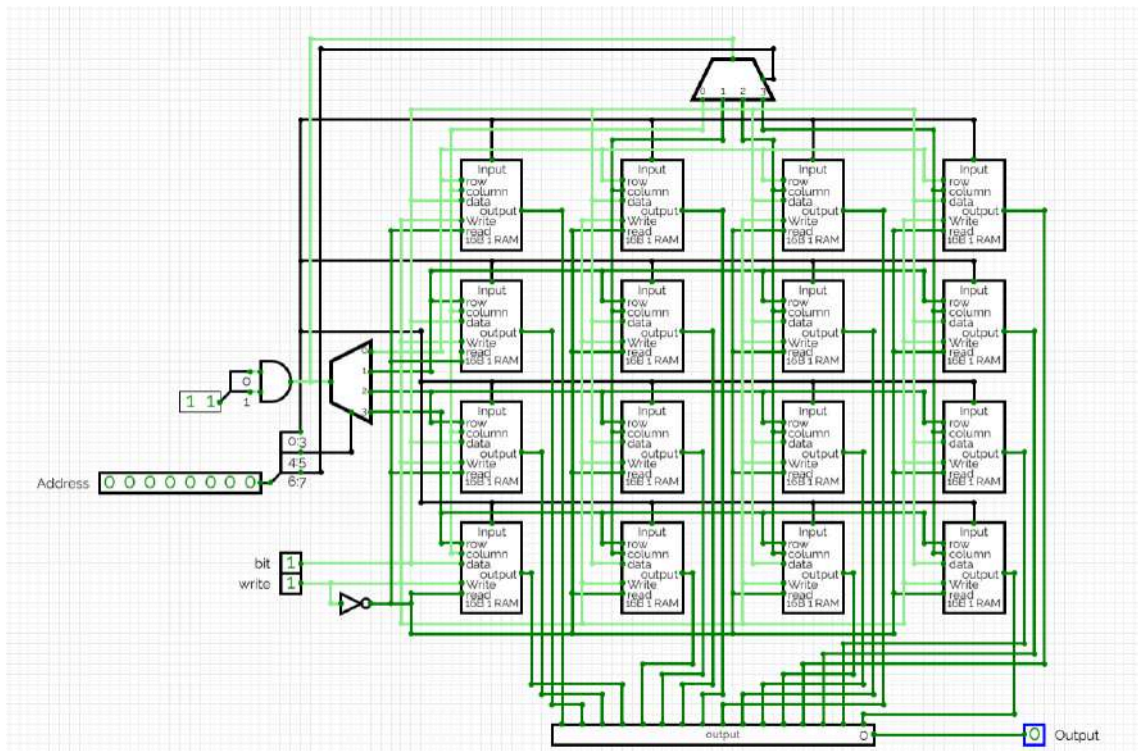


Fig: 256 Bit of SRAM Array using 16 Bit SRAM Array

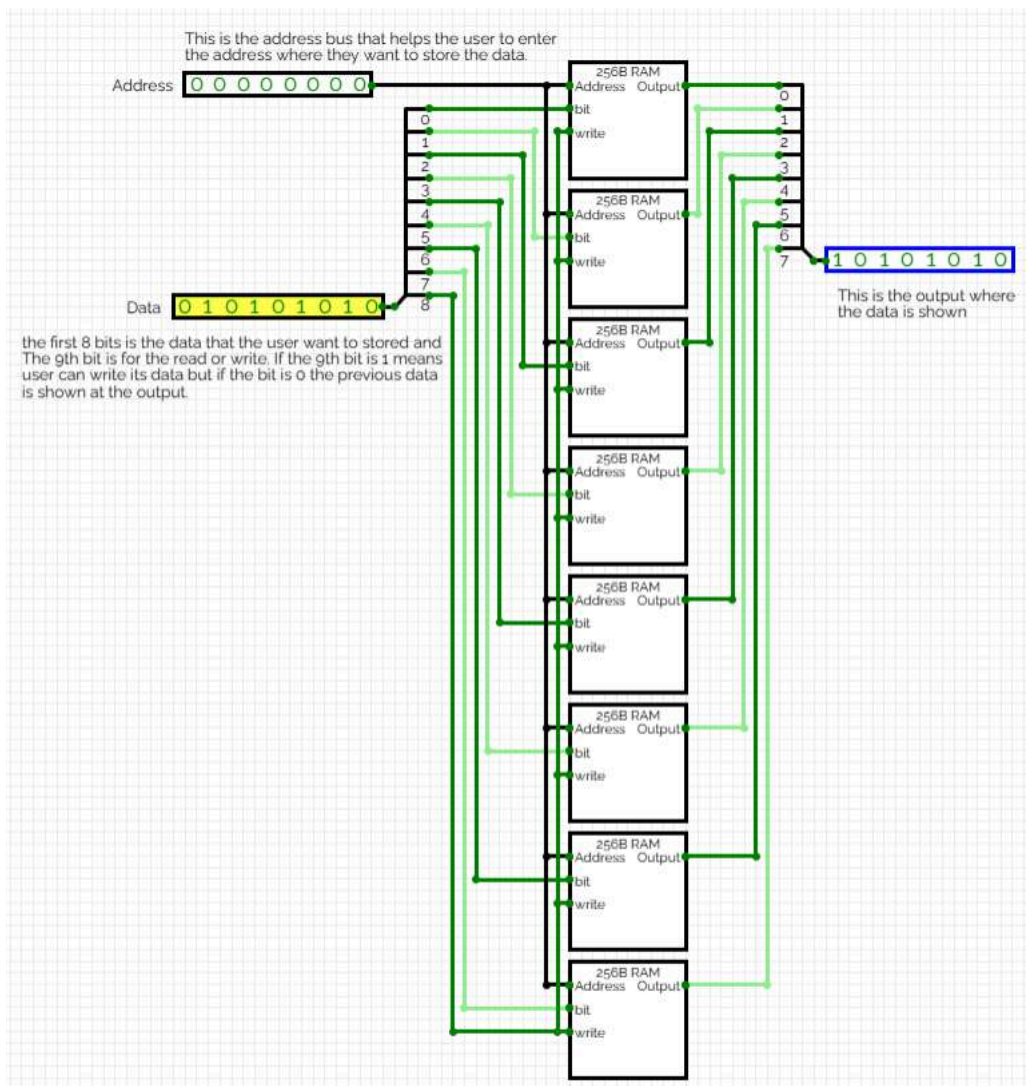


Fig: 8 Bit Data Input Storage SRAM using 256 Bit SRAM Units

3. **Complete Computational System:** All blocks are connected to each other and make a whole working system where all of them interact with each other and exchange the data with each other to give the final output.

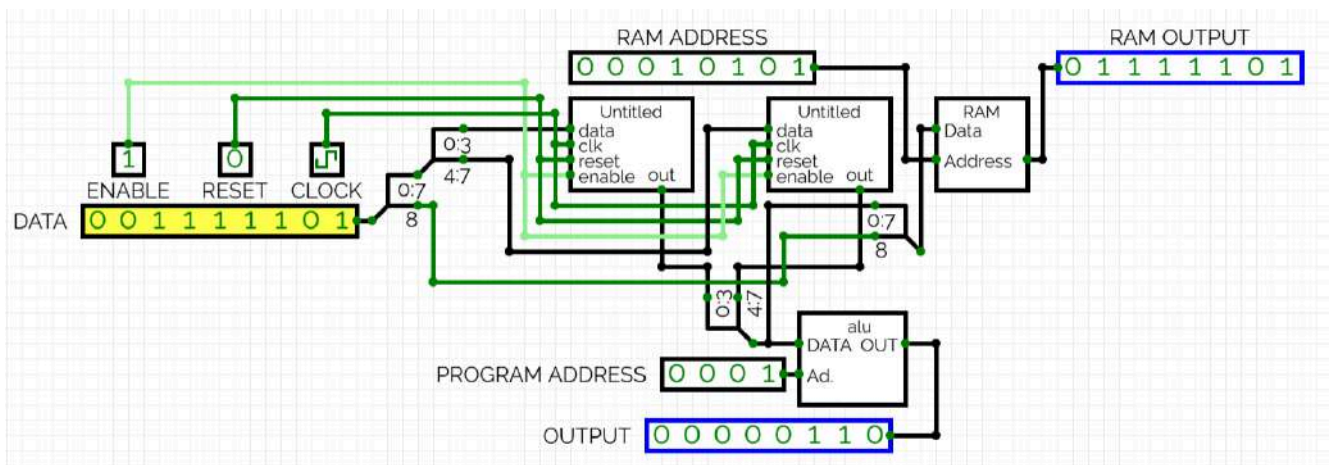


Fig: Complete Computational System

III. 11-Instruction Arithmetic Logic Unit (ALU)

The ALU is the core processing engine, capable of 4 arithmetic and 7 logical operations, selected via a 4-bit instruction opcode.

Opcode	Operation	Description
0000	ADD	4-bit Binary Addition
0001	SUB	2's Complement Subtraction
0010	MUL	Sequential Multiplication
0011	DIV	Sequential Division logic
0100	AND	Bitwise AND
0101	OR	Bitwise OR
0110	NOT	Bitwise Inversion
0111	NAND	Universal Logic NAND
1000	NOR	Universal Logic NOR
1001	XOR	Bitwise Exclusive OR
1010	XNOR	Bitwise Equality Check

IV. Functional Verification & Simulation Results

The system was verified using exhaustive simulation cases in CircuitVerse to ensure timing accuracy between the registers and the RAM array.

Test Case 1: Compute and Store

1. Input: Data A = 0101, Data B = 0011, Opcode = 0000 (ADD).
2. Operation: ALU calculates 1000.
3. Memory Command: Address = 00001111, R/W Bit = 1 (Write).
4. Result: The value 1000 was successfully latched into RAM address 15.

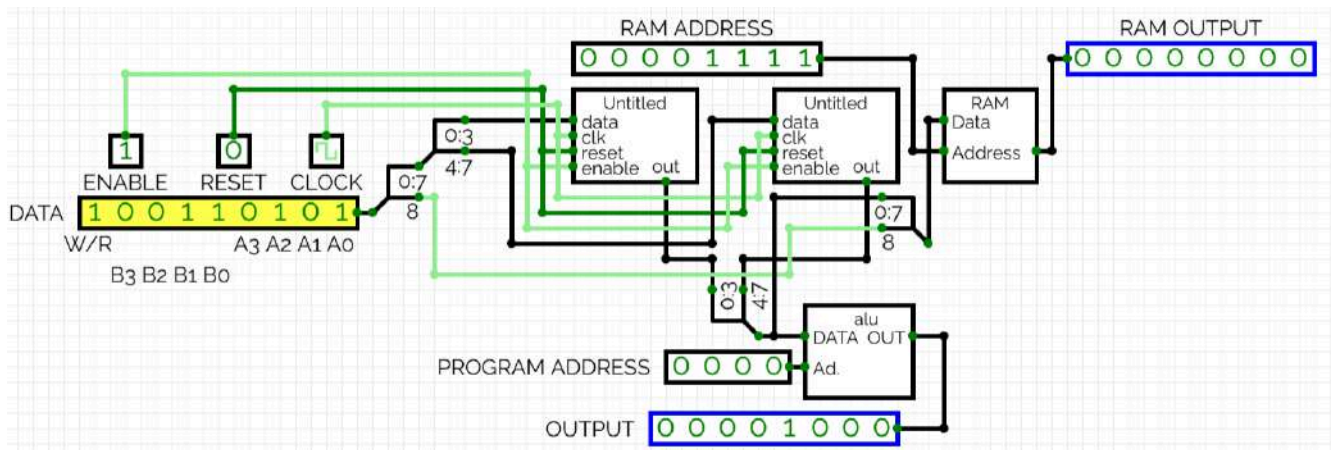


Fig: Simulated Output

Test Case 2: Compute and Store

1. Input: Data A = 0101, Data B = 0010, Opcode = 0010 (MUL).
2. Operation: ALU calculates 1010.
3. Memory Command: Address = 00101101, R/W Bit = 1 (Write) then 0 (Read).
4. Result: The value 1010 was successfully latched into RAM address 45.

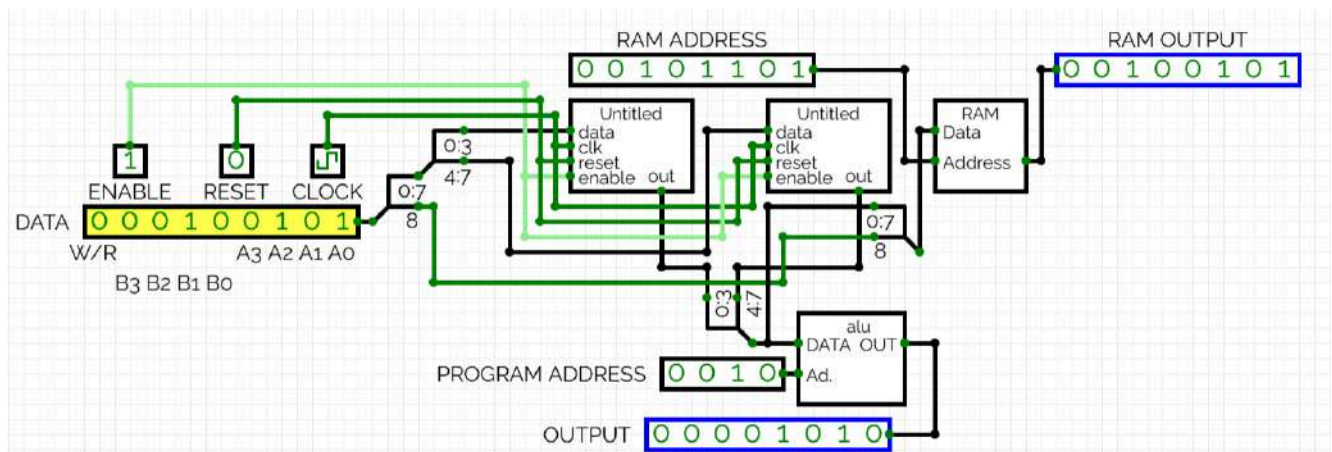


Fig: Simulated Output

Now to read the data whether it is actually hold we are going to check with same test case 2 input but memory address of test case 1 with R/W = 0 (Read).

Result: SRAM Output: 00100101 (Test Case 1 input data Saved at its corresponding address)

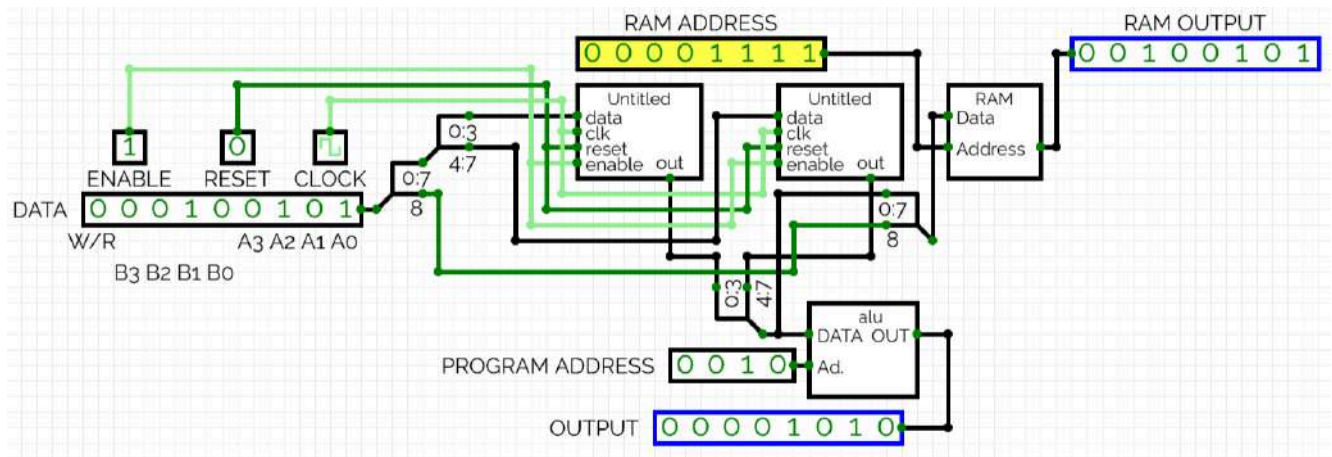


Fig: Simulated Output

Technical Summary of Contributions

- Designed a hierarchical memory structure avoiding pre-built RAM blocks.
- Implemented complex sequential arithmetic (Multiplication/Division) in a purely combinational heavy environment.
- Managed signal integrity across a 8-bit data path with asynchronous Reset/Enable controls.

Project 02: Automated Warehouse Stock Management System

Project Overview

Developed in my **5th Semester**, this project addresses an industrial efficiency problem: automating warehouse stock calculations that were previously managed manually. The system is a hybrid **combinational and sequential logic circuit** designed to track daily stock movement (additions and removals) and calculate net inventory based on the specific number of days in a given month.

I. System Requirements & Logic Design

The system processes three primary data points to automate inventory tracking:

- **Month Input (4-bit):** Identifies the month to determine the cycle length (28, 30, or 31 days).
- **Stock Quantity (3-bit):** Accepts daily stock changes (up to 7 units per day).
- **Date Tracking (5-bit):** A sequential counter that tracks the current day to determine the operation type.

II. Automated Control Logic

The system utilizes complex combinational logic to handle variable month lengths and daily operations:

Condition A: Variable Month Detection

The circuit identifies the end-of-month trigger based on the 4-bit month input:

- **31-Day Logic:** Triggered for months like January (0001) and August (1000).
- **30-Day Logic:** Triggered for April, June, September, and November.
- **28-Day Logic:** Specifically coded for February (0010).

Condition B: Directional Stock Movement

To simulate realistic warehouse flow, the system automates the "Add/Remove" logic based on the date:

- **Odd-Numbered Days:** System performs an **Addition** of the 3-bit quantity to the total.
- **Even-Numbered Days:** System performs a **Subtraction** (Removal) from the total.

III. Sequential Design & Accumulation

The core of the system relies on synchronized counters and registers to maintain data integrity:

1. **Date Counter:** A 5-bit counter that increments with each "Push Button" pulse, representing a new day's entry.

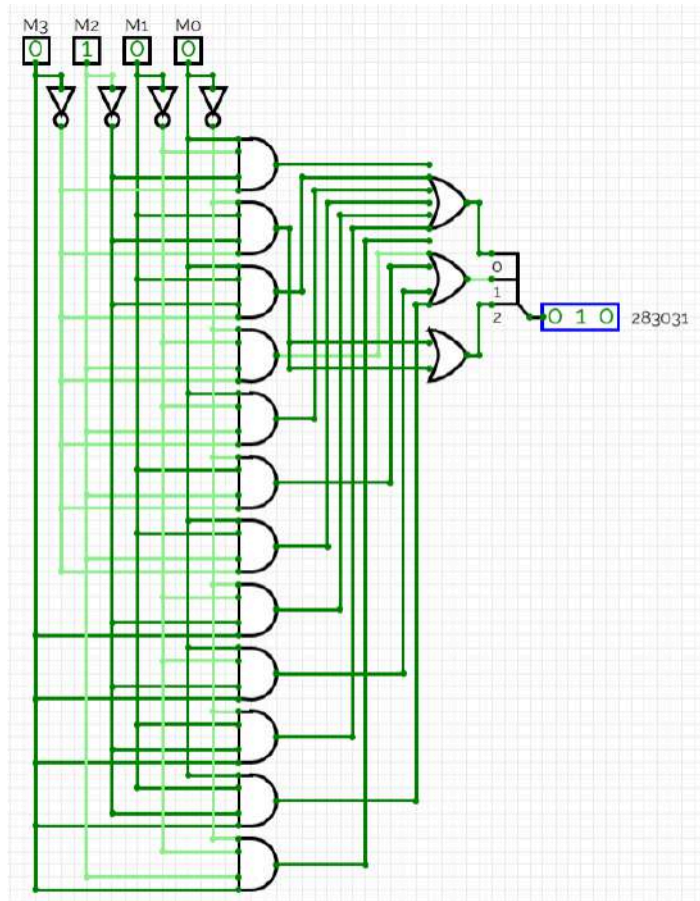


Fig: Decoder for Month Input

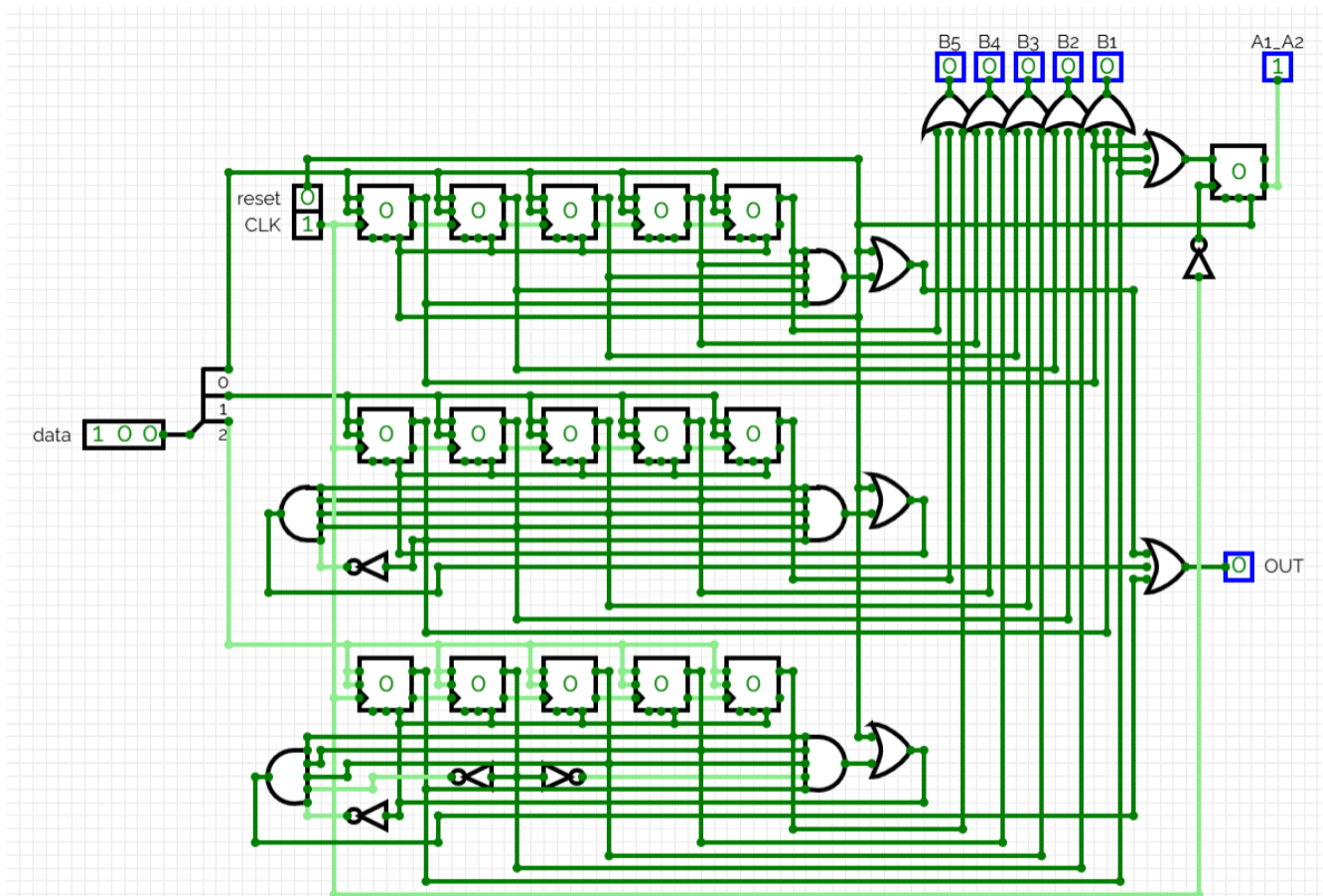


Fig: Counter of 31(top), 30(middle) and 28(bottom) with odd even logic inside it.

2. Arithmetic Accumulator: A 7-bit register that stores the running net total.

To simulate that logic, we are going to add the input data in separate adders for both odd days and even days and then we subtract both at month end and the remaining one gives us the final output.

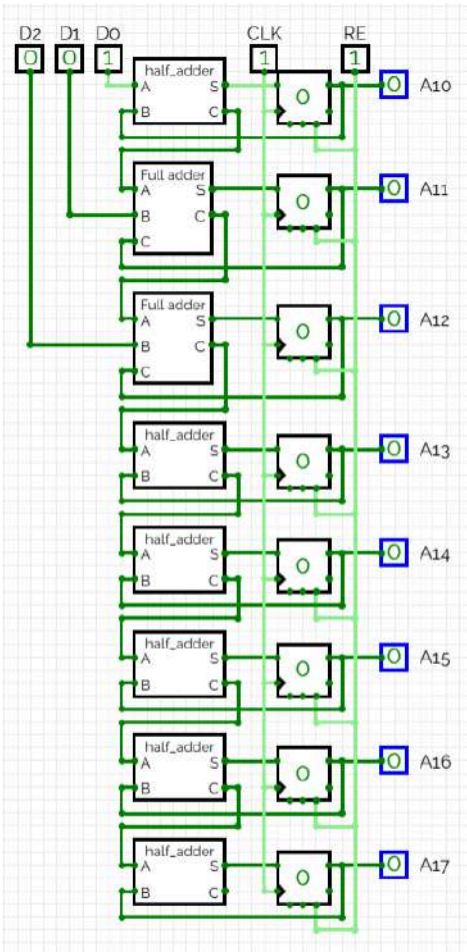


Fig: Adder to add current data with previous data.

Selector to select the adder in which the data is going to be sent according to the date.

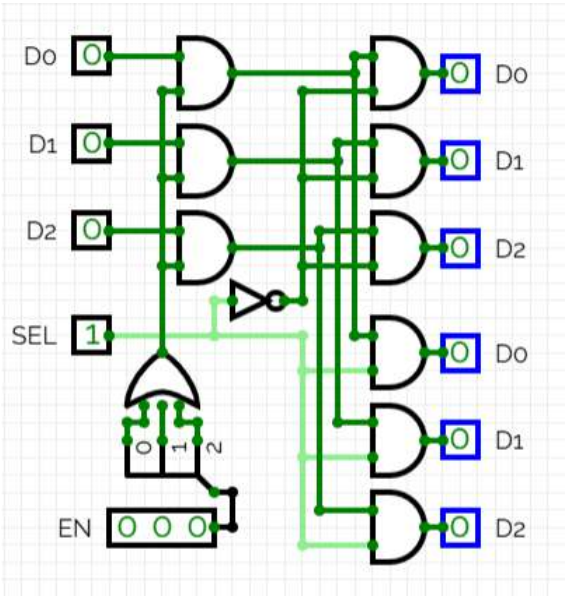


Fig: Selector Unit

3. **Final Calculation:** On the last day of the month, the circuit locks the input and displays the Final Stock Remaining.

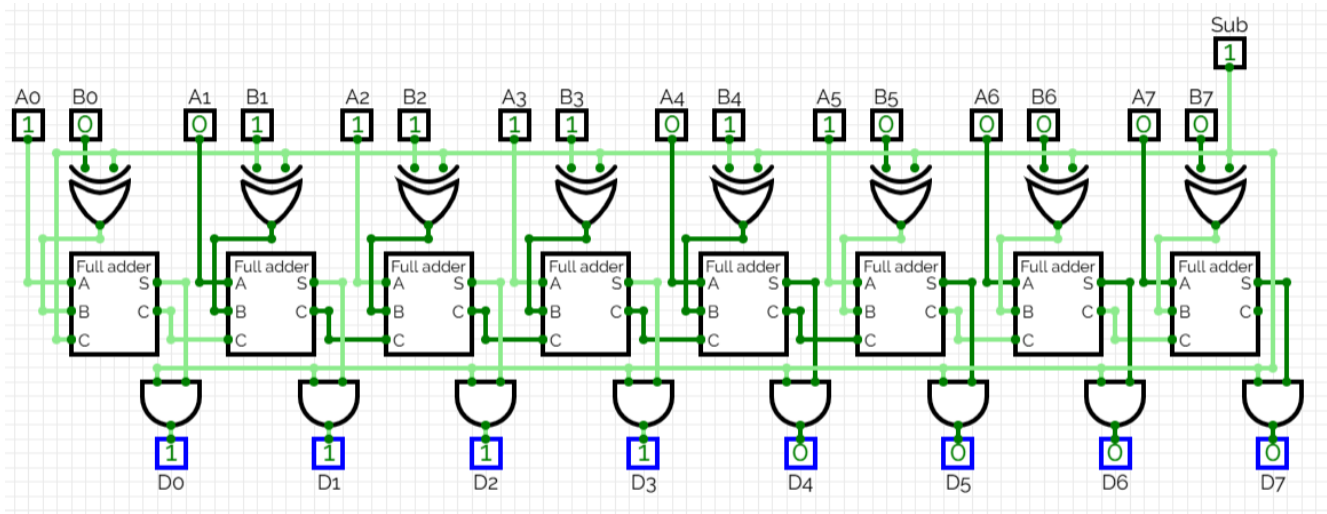


Fig: Subtractor for final output

IV. Implementation & Optimization

- **Component Constraint:** The design was strictly optimized using basic logic gates (AND, OR, XOR), registers, and counters to ensure hardware efficiency.
- **Accuracy:** By using a 7-bit output for the final stock, the system prevents overflow during the monthly accumulation process.

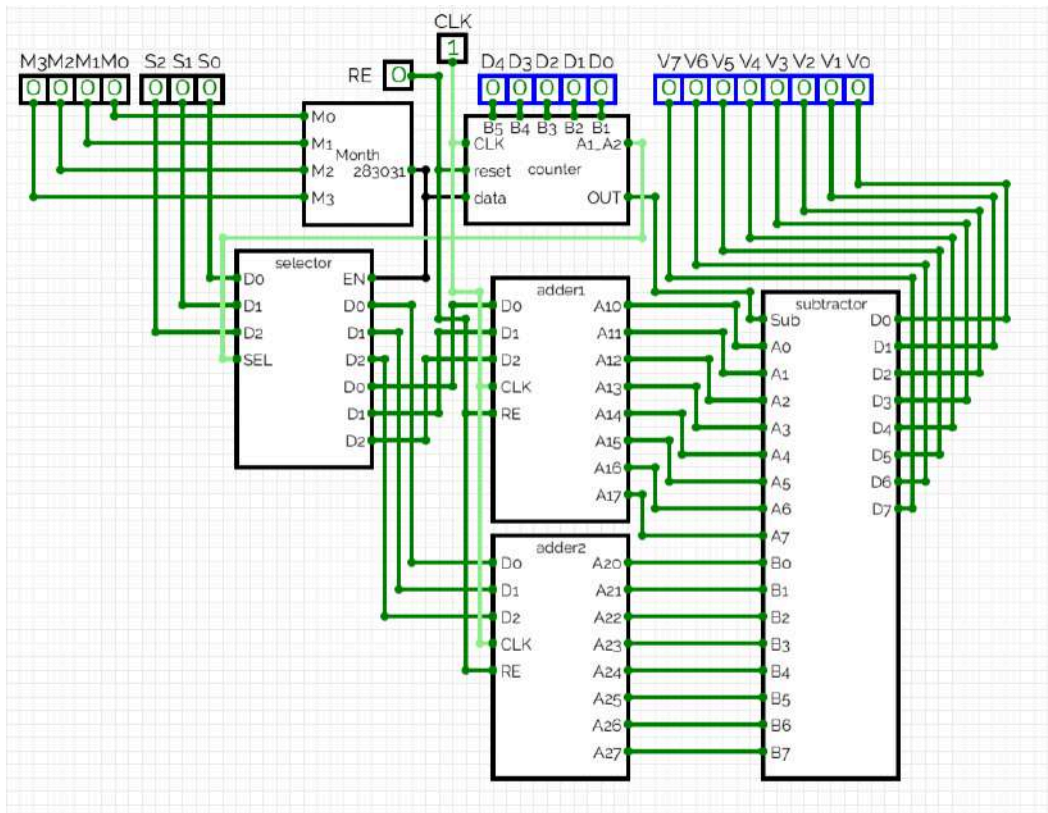


Fig: Final System Design

V. Functional Verification

To verify the accuracy of the sequential accumulation and month-end decoding, the following test scenario was simulated:

- **Test Month:** May (Binary Input: 0101).
- **Cycle Length:** The system automatically identified a 31-day cycle.
- **Constant Daily Input:** A 3-bit stock quantity of 4 was entered every day.

Operational Logic:

- Odd Days (1, 3, 5...31): Stock is added (+4). There are 16 odd days in May.
- Even Days (2, 4, 6...30): Stock is removed (-4). There are 15 even days in May.

Expected Mathematical Result: Total Added: $16 * 4 = 64$

- Total Removed: $15 * 4 = 60$
- Net Remaining: $64 - 60 = 4$.

Actual System Output: On Day 31, the 7-bit binary output displayed 0000100 (Decimal 4), confirming perfect functional alignment.

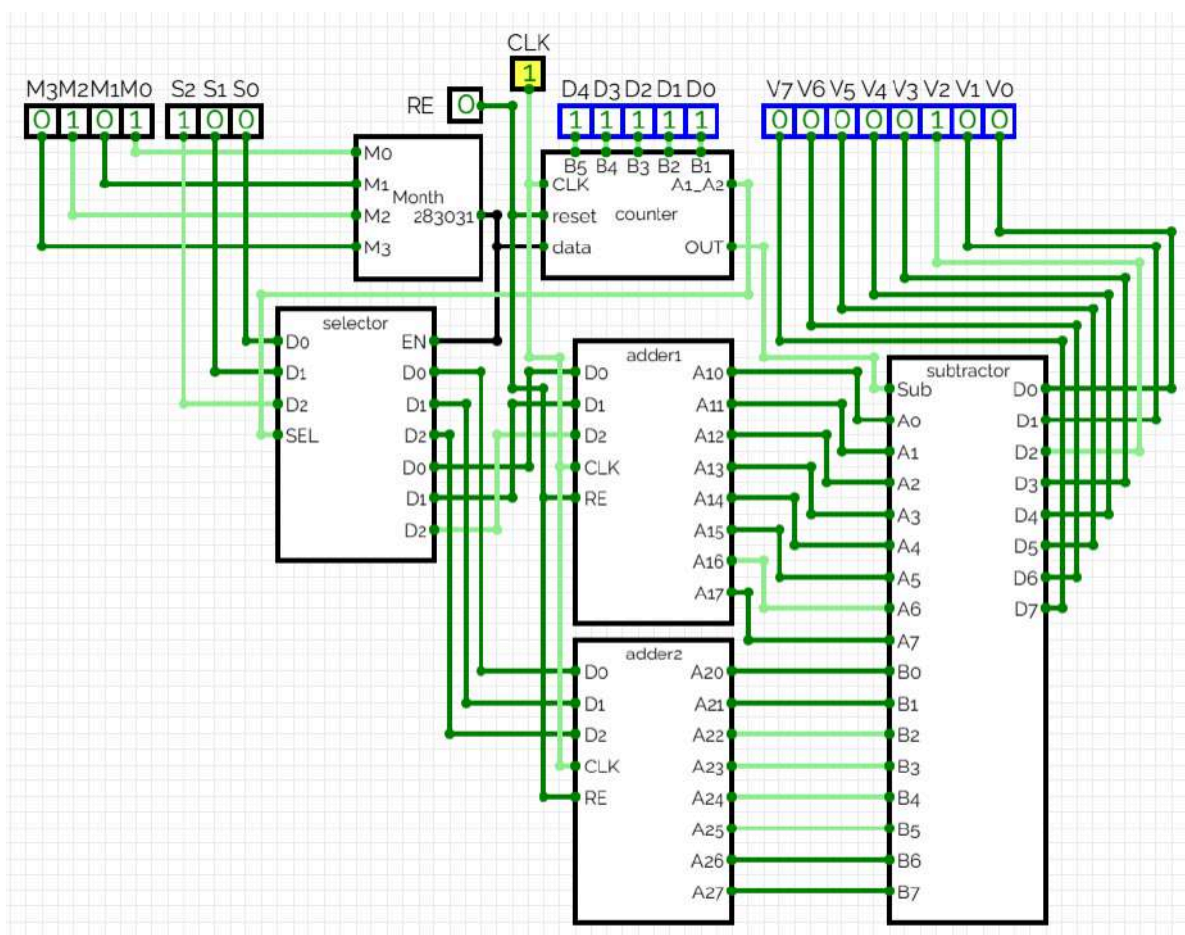


Fig: Simulated Output

Technical Summary of Contributions

- Designed a multi-condition month decoder using combinational logic gates.
- Integrated a control-input system where a single push-button pulse manages both the date increment and the stock calculation.
- Solved the "Real-World Variable" problem by automating different resets for February vs. 31-day months.